

TAAF Engagement (WO-022537, Versent Pty Ltd) — Screening Call Prep

Candidate: Ritesh Agarwal | **Role:** Pr. Engineer / Pr. Consultant

This document consolidates the mock interview questions and the sharpened, evidence-based answers developed during preparation. Use it as a final review before the call, not a script to memorise word-for-word — the goal is to internalise the structure and lead with your own delivered work.

General Delivery Tips (apply throughout)

- **Lead with specifics, not textbook phrases.** Avoid defaulting to "resiliency and scalability" as a generic opener — go straight to the named project and concrete detail.
 - **Land a clear concluding sentence**, even if the middle of an answer is still forming. Don't trail off.
 - **If a detail doesn't come to you, stop at your last confident point** rather than saying "I forgot" out loud.
 - **Own your scope honestly.** "I built the platform; the application team owned decomposition" is a strength, not a weakness, when stated clearly.
 - **Don't lean on "AI confidence" as a validation method** — LLM fluency is not a trust signal. Lean on evidence traceability and human sign-off instead.
-

1. Opening — Relevant Experience

Q: Tell me about your relevant experience and what draws you to this modernization role.

A: 19+ years in IT, with recent focus on cloud-native platform engineering across AWS, Azure, and GCP. Built reusable Amazon EKS platform patterns (Terraform, GitLab CI/CD, IRSA workload identity, autoscaling) for Australia Post; architected an AKS platform with Istio service mesh and GitOps (Argo CD, Helm) for Telstra; led app modernization and platform build with Terraform/GitOps at Deutsche Bank. Recently completed the Claude Certified Architect - Foundations (CCAF) and a Graduate Certificate in AI, which built a strong conceptual grounding in agentic AI — drawing me toward roles like this where platform engineering and GenAI-driven modernization intersect.

2. Multi-Agent Framework Design

Q: What do you understand by a "multi-agent framework" for application modernization, and why might a customer prefer that over a single automation tool?

A: A multi-agent framework decomposes the modernization platform into discrete agents — DevOps, security, application-modernization — each with a narrow mandate, its own guardrails, and its own audit trail, coordinated through clear hand-offs (e.g. the application-modernization agent proposes a categorisation, the security agent validates it against

policy, the DevOps agent only acts once both sign off). This matters for governance: if one agent misbehaves or hallucinates, it doesn't have authority to also touch security policy or deployment pipelines. A single monolithic "do-everything" agent is much harder to govern, test, and explain to a customer's risk/security team — especially in a regulated, multi-tenant context.

Q: If you mapped your GitLab CI/CD, Terraform, and Argo CD/Helm experience onto the DevOps agent, what would it actually be responsible for?

A: Generating/validating Terraform for infrastructure provisioning (as with the reusable EKS patterns at Australia Post), managing the GitOps deployment loop via Argo CD syncing desired state to the cluster (as at Telstra), and enforcing Helm chart standards for consistent application delivery — essentially automating and validating the IaC/CI/CD work already done manually.

3. AI-Powered Application Portfolio Categorisation

Q: How would you approach categorising applications (rehost/replatform/refactor/retire/retain) using GenAI, and how do you make sure recommendations are trustworthy rather than plausible-sounding guesses?

A: Beyond few-shot prompt examples, the categorisation needs to be grounded in real evidence specific to each application: its codebase (language, framework, dependencies), a dependency graph (what it talks to, what talks to it), and operational telemetry (traffic patterns, resource usage, incident history). Every AI-generated recommendation should cite the specific evidence that produced it, and a human — a Principal Consultant or similar — reviews and signs off before anything goes into a client deliverable. Few-shot examples teach the pattern of a good answer; the application's own evidence is what makes a specific recommendation correct rather than plausible.

Q: How would you personally validate an AI-generated categorisation before it went into a client deliverable?

A: (*Correction from initial answer — do not cite "model confidence" as a validation method; LLM confidence is not a calibrated or reliable signal.*) Check that the recommendation cites specific evidence (dependency, telemetry signal, code pattern) rather than an unsupported assertion; spot-check a sample of categorisations against the actual codebase/dependency graph, especially edge cases; require human review/sign-off before anything reaches a client deliverable; flag low-evidence or conflicting-signal cases for manual review rather than trusting the model to self-report uncertainty.

4. Target-State Architecture vs. CCoE Operating Model

Q: What's the difference between the target-state architecture and the CCoE operating model, and why does a customer need both?

A: Target-state architecture is *technical* — the blueprint itself (microservices decomposition, API modernisation, cloud-native design, technology stack) for a given

application or estate. The CCoE operating model is *organisational and governance* — roles and responsibilities, governance processes, standards, and the operating rhythm that keeps things consistent after the AWS ProServe team leaves. A blueprint is a snapshot; without a CCoE, there's no accountability for keeping new applications aligned to the pattern, and the modernisation investment erodes within months. The CCoE is what makes the architecture durable.

Q: What would a CCoE operating model concretely define for this engagement?

A: Roles (who owns platform standards, who approves exceptions, who's accountable when an agent-driven change goes wrong); governance processes (how new applications onboard to standardised patterns, how reference-architecture changes get proposed and approved, how AI agent outputs get periodically reviewed rather than trusted indefinitely); standards and playbooks (the reusable patterns — e.g. reference Terraform patterns, GitOps workflows — documented so new teams can adopt them without reinventing them).

5. Mapping Platform Experience to the Agent Framework

Q: Drawing a direct line from your Australia Post (EKS/Terraform/IRSA) and Telstra (AKS/Istio/GitOps) experience, how would you design the DevOps agent and the security agent?

A:

- **DevOps agent:** Generates/validates Terraform for infrastructure provisioning (Australia Post EKS pattern), manages the GitOps deployment loop via Argo CD syncing desired state to the cluster (Telstra), enforces Helm chart standards for consistent delivery.
- **Security agent:** Enforces IRSA (workload identity) so pods get scoped, least-privilege AWS permissions rather than broad node-level roles; applies CKS-style policy enforcement (Pod Security Standards, network policies) for tenant isolation; uses Istio's mTLS/service-mesh controls for network segmentation between tenant workloads. Critically, the security agent should **enforce as a gate** (block a non-compliant deployment) rather than just report on violations after the fact.

Q: Explain IRSA and Azure Workload Identity, and why namespace/service-account scoping matters over node-level IAM roles.

A: IRSA (IAM Roles for Service Accounts) uses OIDC federation to attach an IAM role directly to a Kubernetes service account, so a pod gets short-lived, scoped AWS credentials for exactly the permissions that role has — pod-level least privilege instead of node-level blanket access. Azure Workload Identity is the AKS equivalent, federating a service account with an Azure AD identity. The risk with node-level IAM roles: Kubernetes bin-packs pods from different applications onto the same node, so every pod on that node inherits the same broad permissions — a compromised pod from App A could reach App B's AWS resources purely by co-location. IRSA/Workload Identity closes that specific multi-tenant security gap.

Q: Walk through, end-to-end, how you'd set up IRSA for a new application team's pod.

A:

1. Associate an OIDC identity provider with the EKS cluster (trust bridge between AWS IAM and the cluster).
 2. Create an IAM role with a trust policy scoped to a specific namespace and service account (not the whole cluster).
 3. Attach the permissions policy to that IAM role — only what the application needs.
 4. Create the Kubernetes service account and annotate it with the IAM role ARN.
 5. The application team references that service account in the pod spec; at pod start, the EKS Pod Identity webhook injects short-lived, scoped credentials automatically — no hardcoded secrets, no broad node-level access.
-

6. Multi-Tenant Security Boundaries & Engagement Governance (Closing Scenario)

Q: A tenant's security agent flags a configuration that would make AWS Information internet-accessible. Walk me through how you'd handle this end-to-end — does it constitute a "Change" requiring approval?

A: If the security agent is designed properly as an enforcing gate, the configuration shouldn't have gone live at all — it should have been blocked pre-deployment. Immediate action: **stop/rollback the change to contain the exposure first**, before anything else — this is a potential compliance issue under clause 4.1 (no internet-accessible AWS Information), not a routine bug ticket. Then: investigate root cause (policy gap? agent bypassed? misconfigured rule?), and correct the underlying policy/pipeline with human validation of the fix — not the agent autonomously patching itself unsupervised.

On the Change question: remediating back to the agreed secure baseline is normal remediation, not a Change. It becomes a Change if the fix needs materially more hours than planned, or if it requires altering the agreed architecture/security framework itself (not just correcting a deviation from it). Regardless of Change status, any incident touching AWS Information exposure risk should be documented and reported transparently, not quietly resolved.

Q: What does "delivery discipline" mean under a fixed Not-to-Exceed engagement, and what would you do if a piece of work needed meaningfully more hours than planned?

A: The Work Order caps hours across resource levels (Pr. Engineer, Pr. Consultant, Staff Consultant). If a task risks exceeding plan, that's a **Change** under the contract — written approval from the Engagement Manager is required **before** proceeding, not after. In practice: flag the overrun risk as early as possible, document the reason clearly (scope creep, unexpected complexity, dependency delay), don't proceed with the extra work until written approval is granted, and keep weekly time entry in PSA Communities accurate throughout so there's a clear audit trail if the Change needs justifying.

7. Migration & Platform Experience (Own It Honestly)

Q: Have you personally been involved in an application/workload migration? Walk through a specific instance.

A: At Deutsche Bank, hosted several applications onto Kubernetes. The trickiest part was dependency discovery — identifying what each application depended on, and profiling CPU/memory usage to correctly size microservice quotas. A key constraint was latency: several applications were trading-related, where even small latency increases could materially impact the business, so CNI selection and node/topology-aware scheduling were key levers used to manage it.

(Note: this dependency-discovery and resource-profiling work is exactly the kind of evidence an AI-powered categorisation agent would need — worth stating explicitly as a bridge between past manual work and this engagement's AI-driven approach.)

Q: Have you led application decomposition from monolith to microservices?

A: *(Own this honestly rather than overclaiming.)* "I haven't personally led an application's domain decomposition into microservices — that's typically been the application/domain team's call. My role has been building and enforcing the underlying Kubernetes and service-mesh platform — Istio for traffic management and resilience, autoscaling, ingress/egress patterns — that makes microservices operable at scale once the boundaries are decided by the application team."

8. Hands-On GenAI / Agentic Experience (Biggest Fitment Risk — Handle Honestly)

Q: Have you personally built, configured, or deployed anything resembling an agentic AI system where an AI component took autonomous action?

A: Lead with the strongest evidence: *"I built a local proxy with a cheap 'judge' LLM that filters and trims context before it reaches GitHub Copilot's model — cut token usage 40–45%."* Mechanism: the proxy intercepts requests to the LLM, identifies that a large proportion of tokens were boilerplate/instruction context rather than the actual query, and uses a cheaper LLM to judge intent and filter what context is actually needed before forwarding the trimmed request to the expensive model. Note honestly: the judge LLM's role was filtering/routing context, not reasoning about or answering the query itself — if it couldn't determine relevant context, it passed the query through as-is rather than guessing.

Also mention: POCs for an AI tutor and AI career coach; formal grounding via CCAF and the Graduate Certificate in AI.

Explicit bridge to make on the call: *"That judge-LLM-as-gatekeeper pattern is structurally the same principle as a security agent gating what a DevOps agent is allowed to act on — cheap agent filters, expensive/privileged agent acts only once validated."*

Honest acknowledgment if pressed further: *"I haven't personally deployed a GenAI-driven autonomous agent in a live engagement — my strongest adjacent evidence is GitOps"*

reconciliation loops (Argo CD detecting drift and self-correcting without human intervention, which is itself a form of autonomous action) and the context-filtering work in my extension. I'd be extending patterns I already know with an LLM reasoning layer, not starting from zero."

Q: What techniques reduce hallucination risk when an LLM makes a judgement call (e.g. categorising an application)?

A: Grounding/RAG — feeding real source documents/code/data at query time rather than relying on training knowledge; few-shot examples for consistent categorisation patterns; lower temperature / constrained output for judgement tasks; human-in-the-loop review before anything reaches a client deliverable; and asking the model to cite its source or say "I don't know" rather than forcing an answer when evidence is insufficient.

9. ECS vs. EKS

Q: Have you worked with ECS specifically, or is your container orchestration experience entirely Kubernetes-based? How would you compare ECS and EKS for a client's workload?

A: "I haven't used ECS directly — my orchestration experience is entirely Kubernetes-based, EKS and AKS specifically. For a simple, single-team, AWS-only workload, ECS is often the pragmatic choice — less operational overhead, no cluster to manage, especially with Fargate. Where I'd push for EKS is when there's a need for portability across clouds, a Kubernetes-native tooling ecosystem (Helm, Istio, Argo CD, CKS-style policy tooling), or the client's teams already have Kubernetes skills — which is the pattern I've built for clients like Australia Post and Telstra."

Trade-off summary:

- **ECS:** simpler to operate, no control-plane management (Fargate = no infrastructure at all), tight native AWS integration, smaller learning curve. Good fit for smaller/simpler, AWS-only workloads.
 - **EKS:** needed when you require multi-cloud/hybrid portability, the broader Kubernetes ecosystem, or the client already has Kubernetes skills/tooling investment. Costs more in operational complexity (node groups, upgrades, add-ons even though AWS manages the control plane).
-

10. Infrastructure as Code — Best Practices

Q: When you set up Terraform for a client engagement, what best practices did you follow to make it reusable, safe, and maintainable?

A: "I maintained a central Terraform module repository, structured as service-scoped modules — EKS, EC2, networking — each independently versioned via git tags so consuming teams could pin to a known-good release rather than always pulling the latest. For state, I used S3 as the remote backend with DynamoDB for state locking to prevent concurrent-apply conflicts, and kept environments — dev, staging, prod — isolated via

separate state files rather than relying on workspaces alone, so a mistake in one environment couldn't touch another. Changes went through CI/CD with a plan step reviewed before apply, particularly for production."

Key elements to cover:

- Central, reusable module repository (avoid teams reinventing Terraform from scratch)
 - Service-scoped modules (EKS, EC2, networking) — separation of concerns
 - Module versioning/tagging — consumers pin to known-good releases
 - S3 remote state + **DynamoDB for state locking** (prevents concurrent-apply corruption)
 - **Environment isolation** via separate state files per environment (dev/staging/prod) — not workspaces alone
 - **Plan-before-apply** in CI/CD, reviewed before production changes
 - Least-privilege IAM for the CI/CD pipeline identity running Terraform
-

11. Agentic Frameworks / AWS AI Services

Q: What AWS services are you aware of for building agentic AI systems — hosting models, orchestrating agents, retrieval/grounding? Have you used them hands-on?

A: "On AWS, these systems are typically built on **Bedrock** as the core managed service for foundation models, **Bedrock Agents** for orchestration, and **OpenSearch** as a vector database for RAG/grounding. I haven't had a live client engagement using Bedrock directly — my knowledge there is from the CCAF certification, the Graduate Certificate in AI from Deakin, and the POC applications I built myself, like the AI tutor and career coach. I know the service landscape, but I'd be applying it hands-on for the first time in a live AWS engagement rather than repeating something I've already delivered."

Prep tip: if time allows before the call, spin up a trivial Bedrock example (single model invocation via console/CLI, or a toy RAG setup with OpenSearch Serverless) to honestly upgrade this from "I know the landscape" to "I've tried it hands-on."

12. Microservices Deployment Pipeline (GitOps)

Q: Walk me through, practically, how a new microservice gets deployed — from a developer merging code to it running on the cluster.

A: Sequenced answer (GitOps/Argo CD flow, as used at Telstra):

1. **Developer merges code** to main after PR review.
2. **CI pipeline triggers** (GitLab CI) — runs tests, builds a container image, pushes to a registry (ECR/ACR).
3. **CI updates the deployment manifest** in the separate GitOps repo (Kubernetes manifests/Helm values), bumping the image tag — Git is the source of truth for desired state, not a person or pipeline touching the cluster directly.

4. **Argo CD detects the change** in the GitOps repo and **reconciles** — pulls the new manifest and applies it to the cluster.
 5. **Rollout happens** per the deployment strategy (rolling update, canary, etc.), often via Helm chart configuration.
 6. If Argo CD detects **drift** (manual out-of-band cluster change), it can auto-correct back to the Git-defined state — a form of autonomous reconciliation, worth linking back to the agentic-pattern discussion (Q8).
-

13. Serverless vs. EKS

Q: When would you recommend serverless (Lambda/Fargate) over full EKS for a modernization project — and when would serverless be the wrong choice?

A: "I'd lean serverless — Lambda or Fargate — for event-driven or intermittent workloads where operational simplicity matters more than fine-grained control: no cluster to manage, scales to zero, pay only for execution. I'd push toward EKS when the workload is long-running or latency-sensitive — like the trading applications at Deutsche Bank, where Lambda's cold-start and execution-time limits would be a problem — or where we need deep control over networking and security via a service mesh, or portability across clouds, which is the pattern I've built for clients like Australia Post and Telstra."

When serverless fits: event-driven/intermittent workloads, variable/unpredictable traffic (scales to zero), low operational overhead priority, Fargate as a middle ground (containers, no node management).

When EKS fits better: long-running/stateful/latency-sensitive workloads (Lambda's 15-min execution limit and cold starts are disqualifying), need for fine-grained networking/security control (CNI, service mesh, CKS-style policy), multi-service interdependent architectures, cross-cloud portability requirements.

Summary: Where You Stand

Strongest ground (lead with confidence):

- EKS/Kubernetes platform engineering — IRSA, least privilege, OIDC mechanics
- Concrete delivered work: Australia Post (EKS), Telstra (AKS/Istio/GitOps), Deutsche Bank (migration, trading-latency constraints)
- Honest, well-scoped answers on platform vs. application ownership
- The token-optimisation VS Code extension — best hands-on GenAI evidence, with a measurable outcome

Genuine risk (handle honestly, don't conceal):

- Limited direct experience deploying autonomous multi-agent GenAI systems in a live engagement. Bridge confidently to GitOps reconciliation loops and the judge-LLM pattern rather than apologising for the gap.

- No hands-on experience with AWS Bedrock, Bedrock Agents, or OpenSearch as a vector store — awareness-level only (CCAF, Grad Cert, personal POCs). Keep this framing consistent with the broader agentic-AI gap above.
- No recent hands-on ECS experience — orchestration background is entirely Kubernetes (EKS/AKS/GKE). Fine to state directly; know the ECS-vs-EKS trade-offs well enough to reason about the choice even without hands-on ECS time.

Habits to watch on the call:

- Don't default to generic phrases ("resiliency and scalability") before giving the real example
- Don't trail off on details you can't recall — stop at your last confident point
- Slow down slightly and land clear concluding sentences early in each answer